UNIVERSIDAD TECNOLÓGICA METROPOLITANA

División de Tecnologías de la Información y Comunicación

PROYECTO INTEGRADOR

AulaSync

Sistema inteligente de gestión y control de acceso mediante cerraduras digitales IoT para laboratorios y espacios académicos

Documento de Investigación, Arquitectura de Software y Avance de Proyecto

Integrantes del Equipo de Investigación

Kevin Arnold Ehrenzweig Aguilar
Edgar Rubén Patrón Salazar
Valeria Monserrat Almeida Osorio
Juan Pablo Aké Soberanis

Mérida, Yucatán · 2026

Índice general

Resumen del Proyecto Integrador iv

1 MARCO CONTEXTUAL Y PROBLEMATIZACIÓN 1
- 1.1 Introducción y Contexto Institucional de la UTM . . . 1
- 1.2 Antecedentes del Control de Acceso Académico . . . 2
- 1.3 Planteamiento Detallado del Problema . . . 2
  - 1.3.1 Ineficiencia Operativa del Modelo Manual de Llaves . . . 3
  - 1.3.2 Vulnerabilidades de Seguridad Física y Copia no Autorizada . . . 3
  - 1.3.3 Ausencia de Trazabilidad y Datos Telemétricos en Tiempo Real . . . 3
  - 1.3.4 Inflexibilidad en la Gestión de Solicitudes Excepcionales . . . 3
- 1.4 Propuesta de Solución: El Ecosistema AulaSync . . . 4
- 1.5 Matriz de Requerimientos del Sistema . . . 5
  - 1.5.1 Requerimientos Funcionales (RF) . . . 5
  - 1.5.2 Requerimientos No Funcionales (RNF) . . . 6
- 1.6 Justificación Técnica, Económica y Social . . . 6
- 1.7 Objetivos de la Investigación . . . 6
  - 1.7.1 Objetivo General . . . 7
  - 1.7.2 Objetivos Específicos . . . 7

2 MARCO TEÓRICO Y ESTADO DEL ARTE 8
- 2.1 Internet de las Cosas (IoT) y Smart Campus . . . 8
- 2.2 Microcontroladores y Comunicaciones Inalámbricas . . . 8
  - 2.2.1 Arquitectura de Hardware Dual-Core Xtensa LX6 (ESP32) . . . 9
  - 2.2.2 Protocolos de Red: Wi-Fi, HTTP/REST y MQTT . . . 9
- 2.3 Autenticación Biométrica Dactilar . . . 10
  - 2.3.1 Algoritmos de Extracción y Comparación de Minucias . . . 10
  - 2.3.2 Sensores Ópticos y Capacitivos (AS608/R307) . . . 10
- 2.4 Contraseñas Dinámicas de Un Solo Uso (OTP) . . . 10
  - 2.4.1 Estándares RFC 4226 (HOTP) y RFC 6238 (TOTP) . . . 10
  - 2.4.2 Modelo de Ventanas de Tolerancia y Efimeridad en AulaSync . . . 11
- 2.5 Actuadores Electromecánicos y Electrónica de Potencia . . . 11
  - 2.5.1 Pestillos Solenoides y Cerraduras Electromagnéticas . . . 11
  - 2.5.2 Relevadores Optoacoplados y Protección con Diodos de Flyback . . . 12
- 2.6 Arquitectura de Software Web y Patrón API REST . . . 12
- 2.7 Bases de Datos Relacionales (Engine MariaDB) . . . 13

2.8 Ciberseguridad Aplicada e Ingeniería de Seguridad 13
2.8.1 Autenticación Federada Single Sign-On (Google OAuth 2.0) 13
2.8.2 Mitigación de Inyecciones SQL y Vulnerabilidades OWASP 13
2.8.3 Mitigación de Replay Attacks y Cifrado TLS/SSL 14

3 DISEÑO DE ARQUITECTURA Y MODELADO DE DATOS 15
3.1 Arquitectura General del Sistema AulaSync 15
3.2 Modelado de Base de Datos Relacional (`aulas_inteligentes`) 16
3.2.1 Diccionario de Datos del Esquema Relacional 16
3.2.2 Definición de Esquema SQL (DDL de Referencia) 17
3.3 Especificación Técnica de la API REST 18
3.4 Diagramas de Secuencia y Lógica de Interacción 19
3.4.1 Flujo 1: Solicitud e Ingreso de OTP Dinámico 19
3.4.2 Flujo 2: Control Remoto Administrativo 20

4 IMPLEMENTACIÓN Y DESARROLLO DEL PROYECTO 21
4.1 Metodología de Desarrollo en Etapas de Madurez 21
4.2 Etapa 1: Fundamentos y Prototipado Base 21
4.3 Etapa 2: Interoperabilidad y Lógica de Negocio Backend 22
4.3.1 El Componente de Validación y Seguridad (`api/helpers.php`) 22
4.3.2 El Endpoint de Apertura de Cerradura (`api/cerradura_abrir.php`) 23
4.4 Etapa 3: Fortalecimiento, Seguridad y Escalabilidad 24

5 PRUEBAS, RESULTADOS Y DISCUSIÓN 25
5.1 Resultados del Prototipo Funcional 25
5.2 Evaluación Comparativa: Sistema Tradicional vs AulaSync 26
5.3 Análisis de Resultados Telemétricos y Trazabilidad 26
5.4 Evaluación de Ciberseguridad e Resiliencia 26

6 CONCLUSIONES Y TRABAJO FUTURO 27
6.1 Conclusiones Generales 27
6.2 Cumplimiento de Objetivos 27
6.3 Recomendaciones para el Despliegue en Producción 28
6.4 Trabajo Futuro y Líneas de Investigación 28

Referencias Bibliográficas 29

Índice de figuras

1.1 Arquitectura conceptual de tres capas del ecosistema AulaSync. Fuente: elaboración propia. 4

2.1 Arquitectura interna de bloques del microcontrolador SoC ESP32. Fuente: elaboración propia. 9
2.2 Representación gráfica esquemática de minucias dactilares (Terminación y Bifurcación). Fuente: elaboración propia. 10
2.3 Esquema de etapas del circuito de potencia y aislamiento galvánico. Fuente: elaboración propia. 12

3.1 Diagrama estructurado de componentes del ecosistema AulaSync. Fuente: elaboración propia. 15
3.2 Mapa de relaciones modularizado entre las 13 tablas del esquema MariaDB. Fuente: elaboración propia. 16

4.1 Diagrama metodológico de las tres etapas de madurez del proyecto AulaSync. Fuente: elaboración propia. 21

5.1 Resultados visuales del prototipo funcional e integración de datos. Fuente: elaboración propia. 25

Índice de tablas

1.1 Matriz comparativa: Modelo Tradicional vs Ecosistema AulaSync. 4

3.1 Catálogo de Endpoints de la API REST de AulaSync. 19

5.1 Métricas de tiempo operativo consumido (promedio por clase). 26

RESUMEN DEL PROYECTO INTEGRADOR

El presente documento expone una investigación exhaustiva y el desarrollo técnico detallado del proyecto integrador **AulaSync**, una plataforma de arquitectura híbrida diseñada para resolver de manera integral la problemática del control de acceso y la administración del agendamiento de espacios académicos (aulas universales, laboratorios de cómputo, talleres especializados y auditorios) en la Universidad Tecnológica Metropolitana (UTM). Históricamente, las instituciones de educación superior han administrado el acceso a sus instalaciones mediante procesos mecánicos y analógicos basados en la custodia manual, entrega física y firma de bitácoras en papel para el préstamo de llaves metálicas. Esta dinámica genera cuellos de botella logísticos en las horas de cambio de clase, provoca la pérdida recurrente de tiempo lectivo efectivo, incrementa los costos por sustitución de combinaciones cerradas ante extravíos y, de manera crítica, carece por completo de mecanismos de trazabilidad e identidad verificable sobre quién ingresó a un espacio, en qué momento exacto y bajo qué justificación institucional.

La solución formulada en **AulaSync** articula un ecosistema tecnológico avanzado que integra tres capas principales: una plataforma web de agendamiento y administración centralizada orientada a roles (Administradores, Docentes y Personal de Mantenimiento); una capa de servicios API REST orientada a microservicios construida en PHP con conexión persistente mediante PDO a una base de datos relacional MariaDB (`aulas_inteligentes`) compuesta por 13 tablas strictly tipadas con restricciones de clave foránea y validación (`CHECK constraints`); y una red perimetral de cerraduras electrónicas inteligentes concebidas como nodos del Internet de las Cosas (IoT). Cada nodo IoT instala en la puerta del recinto un microcontrolador SoC ESP32/ESP8266 con conectividad Wi-Fi 802.11 b/g/n, lector biométrico dactilar óptico/capacitivo (AS608/R307), teclado matricial para ingreso de contraseñas dinámicas de un solo uso (OTP), pantalla de estado y módulo de relevador optoacoplado que acciona la apertura electromecánica de la puerta.

El esquema de ciberseguridad del sistema implementa autenticación Single Sign-On (SSO) federada con Google OAuth 2.0, doble factor de autenticación (2FA) para la apertura física, generación algorítmica de códigos OTP de 6 dígitos con ventana de tolerancia temporal de 15 minutos, mitigación estricta de ataques de repetición (*replay attacks*), prevención de inyecciones SQL mediante parametrización PDO y auditoría telemétrica inmutable en tiempo

real. La investigación adopta una metodología modular estructurada en tres fases de madurez: Fundamentos y Prototipado, Interoperabilidad y Lógica de Negocio, y Fortalecimiento y Escalabilidad. Los resultados obtenidos demuestran un tiempo de respuesta telemétrico inferior a 1.5 segundos por evento de apertura y una erradicación completa de la fricción operativa ligada al préstamo manual de llaves.

Capítulo 1

MARCO CONTEXTUAL Y PROBLEMATIZACIÓN

1.1 Introducción y Contexto Institucional de la UTM

La Universidad Tecnológica Metropolitana (UTM), ubicada en la ciudad de Mérida, Yucatán, es una institución pública de educación superior dedicada a la formación de profesionales técnicos universitarios e ingenieros en áreas de alta exigencia tecnológica, industrial y administrativa. La infraestructura física de la universidad abarca un vasto complejo universitario integrado por múltiples módulos edilicios diseñados para albergar actividades académicas teóricas, prácticas de laboratorio y eventos institucionales.

Dentro del esquema de organización del campus, la infraestructura física se clasifica por módulos de edificación asignados a distintas divisiones y funciones administrativas:

- **Edificio A (Centro de Información y CSAE):** Alberga la biblioteca central, salas de lectura, cubículos de estudio y las oficinas del Centro de Servicios al Estudiante.
- **Edificio B (Rectoría y Control Escolar):** Concentra las oficinas directivas, servicios escolares, coordinación financiera y áreas de atención administrativa principal.
- **Edificio C (Dirección TIC):** Contiene los laboratorios principales de desarrollo de software, salas de servidores, laboratorios de redes y telecomunicaciones y la dirección de carrera de Tecnologías de la Información.
- **Edificio E (Docencia):** Módulo dedicado a aulas magnas y universales de impartición de cátedras teóricas para diversas carreras tecnológicas.
- **Edificio F (Laboratorios Especializados):** Espacio acondicionado para laboratorios pesados, electromecánica, robótica, diseño multimedia y talleres de física y electrónica.

La dinámica operativa diaria exige la asignación y liberación coordinada de decenas de espacios en turnos matutinos, vespertinos y nocturnos. Cada cambio de bloque de clase implica que profesores y alumnos deban ingresar puntualmente a los recintos correspondientes. Sin embargo, la gestión tradicional del acceso a estas áreas ha dependido históricamente de métodos manuales y mecánicos que resultan inadecuados para el volumen operativo y las exigencias de agilidad de una universidad tecnológica moderna.

1.2 Antecedentes del Control de Acceso Académico

El control de acceso en entornos educativos ha evolucionado a lo largo de las últimas décadas desde modelos totalmente físicos hasta sistemas electrónicos aislados. En los inicios de las instituciones universitarias, la seguridad perimetral de las aulas recaía en la figura del intendente o prefecto de edificio, quien poseía un manojo maestro de llaves físicas y abría manualmente cada puerta al inicio de la jornada laboral.

Con el incremento en la matrícula y el número de salones, este modelo se descentralizó hacia el préstamo individualizado de llaves. En dicho esquema, la administración deposita las llaves en una caseta central de control y exige a los docentes firmar un libro de registro (bitácora física en papel) al retirar la llave y devolverla al término de su sesión de clase. Si bien este procedimiento intentó establecer un principio elemental de responsabilidad, introdujo graves vulnerabilidades operativas:

1. **Dependencia del factor humano:** El registro en bitácora suele omitirse durante periodos de alta afluencia, generando lagunas en la información sobre la custodia real de la llave.
2. **Información estática no verificable:** Las firmas manuscritas y la anotación manual de horas de entrada y salida son fácilmente alterables y no proporcionan un registro inmutable.
3. **Falta de integración con la carga académica:** El encargado de ventanilla no cuenta con una herramienta en tiempo real para verificar si el docente que solicita la llave tiene efectivamente una clase programada en esa aula y en esa hora específica.

1.3 Planteamiento Detallado del Problema

La problemática central que motiva el desarrollo del proyecto **AulaSync** radica en la incompatibilidad estructural entre los métodos analógicos de control de acceso y las necesidades operativas de una universidad digital. Esta problemática se desglosa en cuatro dimensiones críticas:

1.3.1 Ineficiencia Operativa del Modelo Manual de Llaves

Durante los cambios de clase (por ejemplo, a las 07:00, 09:00 o 11:00 horas), decenas de profesores se aglomeran simultáneamente en la ventanilla de servicios generales para solicitar o entregar llaves. Esto genera filas prolongadas y demoras que oscilan entre 10 y 20 minutos por docente. Multiplicado por el número de grupos y salones, la universidad pierde cientos de horas lectivas efectivas al mes únicamente en traslados y trámites de entrega de llaves.

1.3.2 Vulnerabilidades de Seguridad Física y Copia no Autorizada

Las llaves metálicas tradicionales de perfil comercial pueden ser duplicadas en cerrajerías externas sin requerir autorización formal. Esto permite la creación de copias no controladas por parte de usuarios no autorizados, posibilitando el ingreso a aulas y laboratorios fuera de horario. Asimismo, cuando una llave maestra o la llave de un laboratorio especializado provisto de equipos de cómputo de alto valor se extravía, la institución enfrenta el dilema de asumir el riesgo de seguridad o incurrir en elevados costos económicos para reemplazar todos los cilindros mecánicos de las puertas afectadas.

1.3.3 Ausencia de Trazabilidad y Datos Telemétricos en Tiempo Real

Cuando ocurre una incidencia dentro de un laboratorio (como la avería de un proyector, el daño a un equipo de cómputo o el extravío de instrumental), la administración universitaria no dispone de un registro digital telemétrico que certifique con precisión matemática quién abrió la puerta, a qué hora exacta se registró la apertura, si el ingreso fue autorizado por la carga académica y en qué momento se cerró el recinto. La falta de evidencia objetiva dificulta la deslindación de responsabilidades y favorece la impunidad ante actos de negligencia o vandalismo.

1.3.4 Inflexibilidad en la Gestión de Solicitudes Excepcionales

Frecuentemente, los docentes requieren espacios adicionales para impartir asesorías, aplicar exámenes extraordinarios o reubicar una clase debido a fallas técnicas en su aula asignada originalmente (como fallas en el aire acondicionado o en el sistema eléctrico). En el modelo tradicional, la solicitud de un aula sustituta exige un trámite burocrático impreso que requiere firmas de aprobación de la dirección de carrera y la coordinación de servicios generales. Este proceso puede demorar días, imposibilitando la reasignación inmediata de espacios que se encuentran físicamente desocupados en ese momento.

Tabla 1.1: Matriz comparativa: Modelo Tradicional vs Ecosistema AulaSync.

Dimensión	Modelo Tradicional (Llave Física)	Ecosistema AulaSync (IoT + Web)
Tiempo de entrega	10–20 min (filas en ventanilla)	Inmediato (¡1.5 s en la puerta)
Trazabilidad	Bitácora en papel (alterable/incompleta)	Log telemétrico inmutable en MariaDB
Mecanismo de acceso	Llave metálica duplicable	OTP dinámico 6 dígitos / Huella dactilar
Seguridad	Nula ante pérdidas de llaves	Autenticación 2FA y Google OAuth 2.0
Gestión de cambios	Trámite burocrático en papel (días)	Solicitud en plataforma web (minutos)
Costo de extravío	Elevado (reemplazo de cerraduras)	Nulo (revocación digital del código)

1.4 Propuesta de Solución: El Ecosistema AulaSync

Para resolver de manera definitiva las deficiencias del modelo analógico, se propone la concepción, diseño e implementación del ecosistema digital **AulaSync**. Dicha solución integra hardware, software y comunicaciones en un modelo perimetral distribuido.

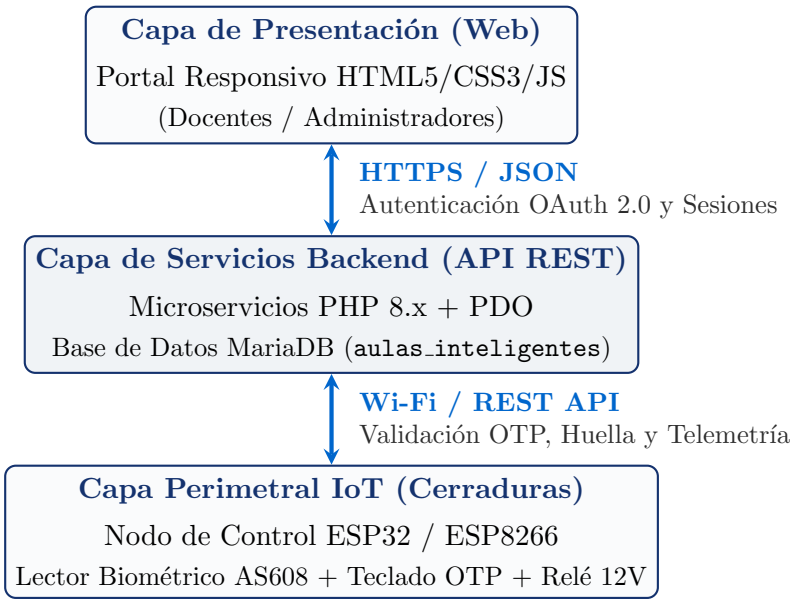


Figura 1.1: Arquitectura conceptual de tres capas del ecosistema AulaSync. Fuente: elaboración propia.

El ecosistema opera bajo una dinámica centralizada donde la base de datos MariaDB actúa como la única fuente de verdad sobre los estados de las aulas (`disponible`, `ocupada`, `mantenimiento`), la carga académica de los profesores y los códigos de acceso autorizados.

Cuando un docente arriba a su aula asignada, puede desbloquear la puerta ingresando el código OTP dinámico generado en su portal web personal o colocando su huella dactilar previamente enrolada en el lector biométrico del nodo IoT. En ambos casos, el microcontrolador valida la autorización consultando a la API REST del servidor y ejecuta la apertura del pestillo electromagnético, registrando simultáneamente la marca de tiempo exacta en la bitácora telemétrica.

1.5 Matriz de Requerimientos del Sistema

El diseño de la plataforma se fundamentó en una especificación rigurosa de requerimientos funcionales y no funcionales que rigen el comportamiento de la solución.

1.5.1 Requerimientos Funcionales (RF)

- **RF-01 (Autenticación SSO):** El sistema debe permitir el inicio de sesión de docentes y administradores mediante sus cuentas institucionales utilizando Google OAuth 2.0.
- **RF-02 (Gestión de Roles RBAC):** La plataforma debe restringir el acceso a los módulos en función del rol asignado (`admin`, `docente`, `mantenimiento`).
- **RF-03 (Consulta de Carga Académica):** El portal docente debe presentar una vista dinámica con las materias, aulas y horarios asignados al profesor para el día en curso.
- **RF-04 (Generación de OTP):** El sistema debe generar contraseñas dinámicas de 6 dígitos numéricos asociadas a un docente y aula específicos, válidas durante la ventana de clase con una tolerancia previa de 15 minutos.
- **RF-05 (Apertura por Teclado Nodal):** El nodo IoT debe recibir el código OTP tecleado en la puerta, enviarlo a la API REST y conmutar el relevador en caso de aprobación.
- **RF-06 (Autenticación Biométrica Presencial):** El nodo IoT debe verificar las plantillas de huella dactilar enroladas y autorizar el acceso presencial sin requerir interacción web.
- **RF-07 (Control Remoto Administrativo):** El panel de administración debe permitir la apertura o cierre manual a distancia de cualquier aula registrada.
- **RF-08 (Solicitud de Aulas Excepcionales):** Los docentes deben poder solicitar cambios de aula o asignaciones adicionales justificadas desde la plataforma web.
- **RF-09 (Resolución de Solicitudes):** El Administrador debe visualizar las solicitudes pendientes y aprobarlas o rechazarlas con actualización inmediata de estado.

- **RF-10 (Bitácora Telemétrica inmutable):** Cada evento de acceso (exitoso o denegado) debe quedar registrado en la tabla `accesos` con sello temporal e ID de usuario.

1.5.2 Requerimientos No Funcionales (RNF)

- **RNF-01 (Tiempo de Respuesta en Puerta):** El proceso completo de lectura de huella/OTP, consulta REST y activación del relevador no debe exceder los 1.5 segundos.
- **RNF-02 (Seguridad de Sesiones y Datos):** Las sesiones deben expirar automáticamente y los parámetros de consulta deben ser sanitizados con PDO para prevenir inyecciones SQL.
- **RNF-03 (Efimeridad y Mitigación de Replay):** Cada código OTP debe ser de un solo uso (`usado = 1`) e invalidarse inmediatamente tras la apertura.
- **RNF-04 (Disponibilidad del Servicio):** La API REST debe garantizar una disponibilidad del 99.5 % sostenida sobre la red local institucional.
- **RNF-05 (Interfaz Responsiva UI/UX):** El diseño web debe adaptarse de forma fluida a dispositivos móviles, tabletas y computadoras de escritorio.

1.6 Justificación Técnica, Económica y Social

El desarrollo del proyecto **AulaSync** se justifica de manera clara desde tres perspectivas complementarias:

- **Justificación Técnica:** Promueve la adopción de tecnologías emergentes de Internet de las Cosas (IoT), arquitectura de microservicios, bases de datos relacionales avanzadas y mecanismos modernos de autenticación en la infraestructura física de la UTM, transformando el campus en un entorno inteligente y auditable.
- **Justificación Económica:** Elimina los gastos recurrentes asociados al duplicado de llaves metálicas, reemplazo de cerraduras por extravío y mantenimiento de chapas mecánicas dañadas, optimizando el presupuesto operativo universitario.
- **Justificación Social y Académica:** Devuelve a la comunidad universitaria cientos de horas lectivas perdidas en trámites burocráticos, mejorando la puntualidad de las clases y fortaleciendo la cultura de responsabilidad y cuidado sobre los activos tecnológicos universitarios.

1.7 Objetivos de la Investigación

1.7.1 Objetivo General

Diseñar, desarrollar e implementar un sistema inteligente de gestión, agendamiento y control de acceso a espacios académicos denominado **AulaSync**, compuesto por una plataforma web de administración centralizada y una red de cerraduras digitales (IoT) conectadas vía Wi-Fi, con el propósito de automatizar la apertura de puertas mediante autenticación biométrica y contraseñas dinámicas OTP, erradicar el préstamo manual de llaves mecánicas y garantizar la trazabilidad telemétrica exacta del uso de la infraestructura universitaria en la Universidad Tecnológica Metropolitana.

1.7.2 Objetivos Específicos

1. Diseñar e implementar el esquema relacional de base de datos MariaDB (`aulas_inteligentes`) compuesto por 13 tablas estrictamente tipadas con restricciones de clave foránea e integridad referencial.

2. Desarrollar una API REST modular en PHP utilizando PDO para orquestar la comunicación bidireccional entre la interfaz web, el servidor de base de datos y la red de cerraduras electrónicas.

3. Programar e integrar el firmware para microcontroladores SoC ESP32/ESP8266 capaces de procesar lectura biométrica dactilar, lectura de teclados matriciales y activación de relevadores electromecánicos.

4. Construir una plataforma web responsiva con paneles diferenciados por rol (Administradores, Docentes y Mantenimiento) para la gestión de carga académica, solicitudes de aula y visualización de telemetría.

5. Aplicar mecanismos de ciberseguridad perimetral y lógica basados en Google OAuth 2.0, validación de tipos estrictos, prevención de inyecciones SQL y mitigación de ataques de repetición.

Capítulo 2

MARCO TEÓRICO Y ESTADO DEL ARTE

El desarrollo del ecosistema **AulaSync** se fundamenta en un marco de conocimiento multidisciplinario que abarca el Internet de las Cosas (IoT), la ingeniería de software web, el diseño de bases de datos relacionales, la biometría aplicada, la electrónica de potencia y la ciberseguridad perimetral.

2.1 Internet de las Cosas (IoT) y Smart Campus

El paradigma del Internet de las Cosas (*Internet of Things*, IoT) describe la interconexión digital de objetos cotidianos a través de redes informáticas, dotándolos de capacidad de cómputo, sensores para percibir su entorno y actuadores para ejecutar acciones físicas. En el contexto educativo, el concepto evoluciona hacia el paradigma de *Smart Campus* (Campus Inteligente), donde la infraestructura edilicia interactúa de forma autónoma con estudiantes y docentes para optimizar el consumo energético, la seguridad perimetral y la disponibilidad de espacios.

En el sistema **AulaSync**, cada puerta de un laboratorio o aula se conceptualiza como un nodo perimetral IoT inteligente. El nodo percibe la presencia del usuario mediante un sensor de huella o un teclado, transmite la telemetría a través de la infraestructura Wi-Fi institucional y ejecuta el desbloqueo electromecánico tras la aprobación centralizada del servidor.

2.2 Microcontroladores y Comunicaciones Inalámbricas

2.2.1 Arquitectura de Hardware Dual-Core Xtensa LX6 (ESP32)

Para la inteligencia local de los nodos se seleccionó la plataforma SoC (*System on Chip*) ESP32 desarrollada por Espressif Systems. El ESP32 integra un microprocesador Xtensa dual-core de 32 bits LX6 con una frecuencia de reloj de hasta 240 MHz, 520 KB de memoria SRAM interna, conectividad Wi-Fi 802.11 b/g/n integrada y módulo Bluetooth v4.2 BR/EDR y BLE.

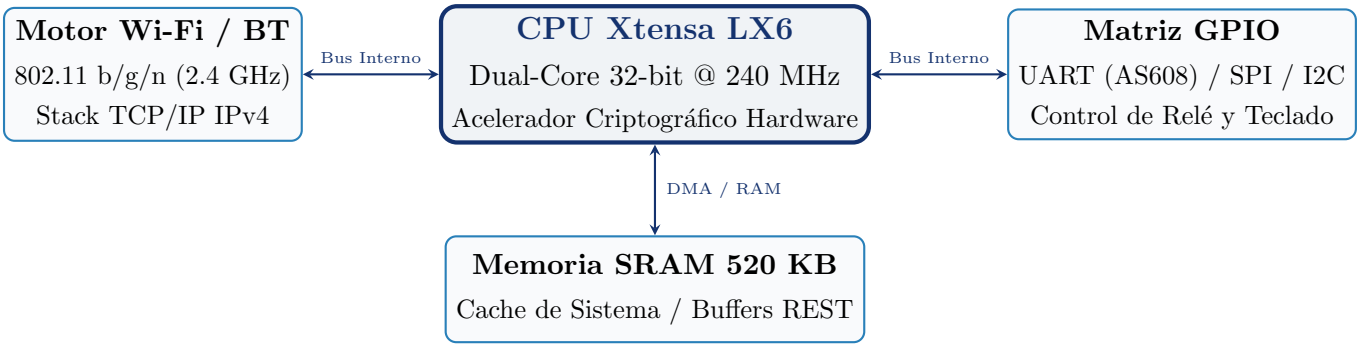


Figura 2.1: Arquitectura interna de bloques del microcontrolador SoC ESP32. Fuente: elaboración propia.

Las ventajas operativas del ESP32 radican en su bajo consumo energético en modos de reposo (*deep sleep*), su alta tolerancia a interferencias electromagnéticas y su amplio soporte para protocolos de pila TCP/IP completos con cifrado hardware por acelerador criptográfico integrado (AES, SHA-2, RSA).

2.2.2 Protocolos de Red: Wi-Fi, HTTP/REST y MQTT

La integración de nodos IoT en redes universitarias requiere seleccionar el protocolo de comunicación adecuado. Se evaluaron principalmente dos enfoques:

- **MQTT (Message Queuing Telemetry Transport):** Protocolo ligero basado en el patrón Publicación/Suscripción (*Pub/Sub*) sobre TCP/IP. Requiere un intermediario (*Broker* como Mosquitto) y es ideal para el envío masivo de telemetría de bajo ancho de banda.
- **HTTP/REST (Representational State Transfer):** Protocolo sin estado basado en peticiones cliente-servidor mediante los métodos estándar HTTP (GET, POST, PUT, DELETE) e intercambio de objetos estructurados en formato JSON.

Para el proyecto **AulaSync** se implementó la arquitectura **HTTP/REST**. Esta decisión se justifica en que la validación de contraseñas OTP y la verificación de permisos requieren un modelo síncrono de Petición-Respuesta (*Request-Response*) donde el nodo necesita confirmar de inmediato si el acceso fue otorgado antes de activar el relevador.

2.3 Autenticación Biométrica Dactilar

2.3.1 Algoritmos de Extracción y Comparación de Minucias

La biometría dactilar es una de las técnicas de identificación de personas más robustas y aceptadas. Se basa en el principio de inmutabilidad y diversidad de las crestas papilares presentes en la epidermis de los dedos.

El procesamiento digital de huellas no almacena la imagen fotográfica directa de la yema del dedo, sino que extrae una plantilla matemática reducida (*template*) a partir de los puntos singulares denominados **minucias**. Las minucias principales son:

- **Terminaciones de cresta:** Puntos donde una línea de la huella finaliza abruptamente.
- **Bifurcaciones:** Puntos donde una cresta se divide en dos ramas divergentes.

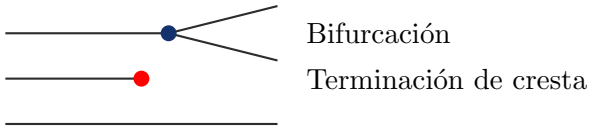


Figura 2.2: Representación gráfica esquemática de minucias dactilares (Terminación y Bifurcación). Fuente: elaboración propia.

Durante la etapa de verificación $(1:1)$ o identificación $(1:N)$, el sensor compara el vector de coordenadas (x, y, θ) de las minucias capturadas contra las plantillas guardadas en memoria mediante algoritmos de alineación espacial y puntuación de coincidencia (*matching score*).

2.3.2 Sensores Ópticos y Capacitivos (AS608/R307)

En los nodos IoT de **AulaSync** se emplean módulos lectores como el **AS608**. Este dispositivo integra un prisma óptico de alta resolución, una fuente de iluminación LED azul y un DSP (*Digital Signal Processor*) dedicado que ejecuta la extracción de minucias y el almacenamiento de plantillas en una memoria EEPROM interna accesible vía interfaz serie UART (TTL 3.3V).

2.4 Contraseñas Dinámicas de Un Solo Uso (OTP)

2.4.1 Estándares RFC 4226 (HOTP) y RFC 6238 (TOTP)

Las contraseñas de un solo uso (*One-Time Passwords*) son mecanismos credenciales concebidos para combatir la reutilización no autorizada de contraseñas estáticas. Los dos

estándares universales normados por la IETF son:

1. **HOTP (HMAC-Based OTP, RFC 4226):** Genera un código basado en un contador incremental C y una clave secreta compartida K aplicando la función hash HMAC-SHA-1:
$$\text{HOTP}(K, C) = \text{Truncate}(\text{HMAC-SHA-1}(K, C)) \quad (\text{mód } 10^d)$$
2. **TOTP (Time-Based OTP, RFC 6238):** Evolución del HOTP que sustituye el contador incremental C por una variable temporal T calculada a partir del tiempo UNIX actual t, un tiempo inicial T_0 y un intervalo de paso X (usualmente 30 o 60 segundos):
$$T = \left\lfloor \frac{t - T_0}{X} \right\rfloor$$

2.4.2 Modelo de Ventanas de Tolerancia y Efimeridad en AulaSync

En el ecosistema **AulaSync**, el algoritmo de OTP se adapta a las reglas de negocio académicas. Cuando un docente solicita su código de apertura desde la aplicación web:

- El servidor verifica la tabla `carga_academica` para confirmar que el profesor tiene una clase asignada en el día de la semana y rango de horas actual.
- Se calcula una ventana de tolerancia paramétrica de 15 minutos previos al inicio formal de la clase (definida por la constante `OTP_TOLERANCIA_MIN` en `config.php`).
- Se genera un PIN numérico de 6 dígitos (`OTP_LONGITUD = 6`) almacenado en la tabla `otp_codigos` con un periodo de expiración estricto de 5 minutos (`OTP_VIGENCIA_MIN = 5`).
- El código es estrictamente efímero: al ejecutarse la primera apertura válida, la base de datos ejecuta el cambio de estado `usado = 1`, invalidando el código contra cualquier intento posterior de apertura.

2.5 Actuadores Electromecánicos y Electrónica de Potencia

2.5.1 Pestillos Solenoides y Cerraduras Electromagnéticas

El actuador electromecánico es el dispositivo responsable de realizar el trabajo físico de liberar el mecanismo de bloqueo de la puerta. Se utilizan comúnmente dos tipos de dispositivos:

- **Pestillo de Solenoide (12V DC):** Dispositivo compuesto por un émbolo de acero impulsado por una bobina electromagnética. Al energizarse, la bobina retrae el émbolo venciendo la fuerza de un resorte interno y permitiendo el paso. Al desenergizarse, el resorte regresa automáticamente el émbolo a la posición de bloqueo.
- **Cerradura Electromagnética (Electroimán de 280 kg):** Placa electromagnética fijada en el marco de la puerta que atrae una placa de armadura de hierro montada en la hoja de la puerta mientras recibe corriente eléctrica (configuración *fail-safe*).

2.5.2 Relevadores Optoacoplados y Protección con Diodos de Flyback

La interfaz de potencia entre las salidas digitales de bajo voltaje del microcontrolador ESP32 (3.3V DC / 12 mA máx) y la carga inductiva del actuador (12V DC / 1A) se resuelve mediante un módulo de relevador (*relay*) optoacoplado.

El optoacoplador utiliza un LED emisor de infrarrojos y un fototransistor receptor internos para aislar galvánicamente la etapa lógica de control frente a picos de voltaje originados en la etapa de potencia. Asimismo, en paralelo con la bobina del actuador se instala un **diodo de desmagnetización o flyback** (1N4007) polarizado en inversa. Este diodo proporciona un camino seguro para la disipación de la fuerza electromotriz de retorno (f.e.m. inductiva) generada cuando el relevador se desactiva, protegiendo al microcontrolador de sobretensiones transitorias destructivas.

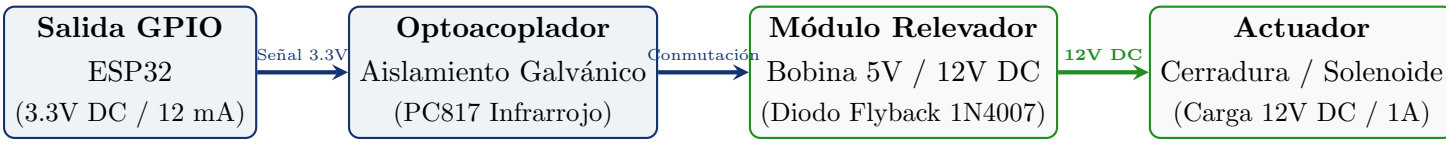


Figura 2.3: Esquema de etapas del circuito de potencia y aislamiento galvánico. Fuente: elaboración propia.

2.6 Arquitectura de Software Web y Patrón API REST

La arquitectura de software de la plataforma **AulaSync** se estructura siguiendo el estilo arquitectónico **REST (Representational State Transfer)** enunciado por Roy Fielding. Bajo este esquema:

- Los recursos de la aplicación (usuarios, aulas, reservas, accesos) se identifican mediante URIs unívocas.
- Las operaciones sobre los recursos se realizan mediante verbos estándar HTTP: `GET` (consulta), `POST` (creación), `PUT` (actualización) y `DELETE` (eliminación).

- La transferencia de estado se ejecuta utilizando paquetes de datos estructurados en formato **JSON (JavaScript Object Notation)** con codificación de caracteres `utf8mb4`.

El código backend se construyó en **PHP 8.x** utilizando la extensión **PDO (PHP Data Objects)**. PDO proporciona una capa de abstracción de acceso a datos independiente del motor de base de datos, garantizando la portabilidad y habilitando el uso de sentencias preparadas (*prepared statements*) con vinculación segura de parámetros.

2.7 Bases de Datos Relacionales (Engine MariaDB)

El motor de base de datos relacional seleccionado es **MariaDB 10.x**, un tenedor (*fork*) de código abierto de MySQL ampliamente reconocido por su alto rendimiento en entornos transaccionales OLTP, soporte completo para motores de almacenamiento InnoDB con cumplimiento de propiedades ACID (Atomicidad, Consistencia, Aisolamiento y Durabilidad) y capacidad para manejar eficientemente índices B-Tree en tablas con elevado volumen de registros.

El esquema relacional diseñado para **AulaSync** aplica una normalización estricta en Tercera Forma Normal (3FN), complementada con restricciones explícitas de integridad referencial (`FOREIGN KEY`) y restricciones de comprobación de tipo (`CHECK constraints`) implementadas a nivel de motor de datos.

2.8 Ciberseguridad Aplicada e Ingeniería de Seguridad

La seguridad en un sistema que controla accesos a infraestructura física perimetral demanda una estrategia de defensa en profundidad:

2.8.1 Autenticación Federada Single Sign-On (Google OAuth 2.0)

Para la capa de identidad web se integró el protocolo estándar industrial **OAuth 2.0** a través de Google Identity Services. Este mecanismo permite a los docentes y administradores autenticarse utilizando sus credenciales institucionales corporativas (`@universidad.edu`). El servidor backend valida el token de autenticación (`google_sub`) y genera una sesión segura almacenando el token unívoco en la tabla `sesiones`.

2.8.2 Mitigación de Inyecciones SQL y Vulnerabilidades OWASP

De acuerdo con las clasificaciones del *OWASP Top 10*, la Inyección SQL (A03:2021-Injection) representa uno de los mayores riesgos para la integridad de los datos. En **AulaSync**, la totalidad de las interacciones con MariaDB se realizan a través del componente

centralizado de helpers (`api/helpers.php`), el cual ejecuta funciones de validación estricta de tipo (`v_entero`, `v_digitos`, `v_texto`) y fuerza el uso exclusivo de consultas parametrizadas PDO con marcadores de posición posicionales o nombrados.

2.8.3 Mitigación de Replay Attacks y Cifrado TLS/SSL

Un ataque de repetición (*Replay Attack*) consiste en que un atacante capture un paquete de red legítimo (como una petición HTTP con un código de apertura) y lo retransmita posteriormente para desbloquear la puerta sin autorización.

AulaSync mitiga esta vulnerabilidad mediante tres líneas de defensa:

1. Cifrado de canal perimetral mediante protocolo **TLS 1.3 / SSL** (HTTPS), impidiendo la lectura del tráfico Wi-Fi en tránsito.
2. Efimeridad del código OTP: una vez consumido un código en `api/cerradura_abrir.php`, la instrucción SQL actualiza inmediatamente `usado = 1`. Cualquier retransmisión posterior del mismo código encuentra el registro marcado como utilizado y retorna un código de error HTTP `409 Conflict`.
3. Marcas de tiempo de expiración estricta (`expira_en`): si un código no se utiliza dentro de sus 5 minutos de vigencia, el servidor deniega el acceso con código HTTP `410 Gone`.

Capítulo 3

DISEÑO DE ARQUITECTURA Y MODELADO DE DATOS

3.1 Arquitectura General del Sistema AulaSync

El sistema **AulaSync** se concibe como una solución integral distribuida que articula la gestión administrativa web con la operación electromecánica en las puertas del campus.

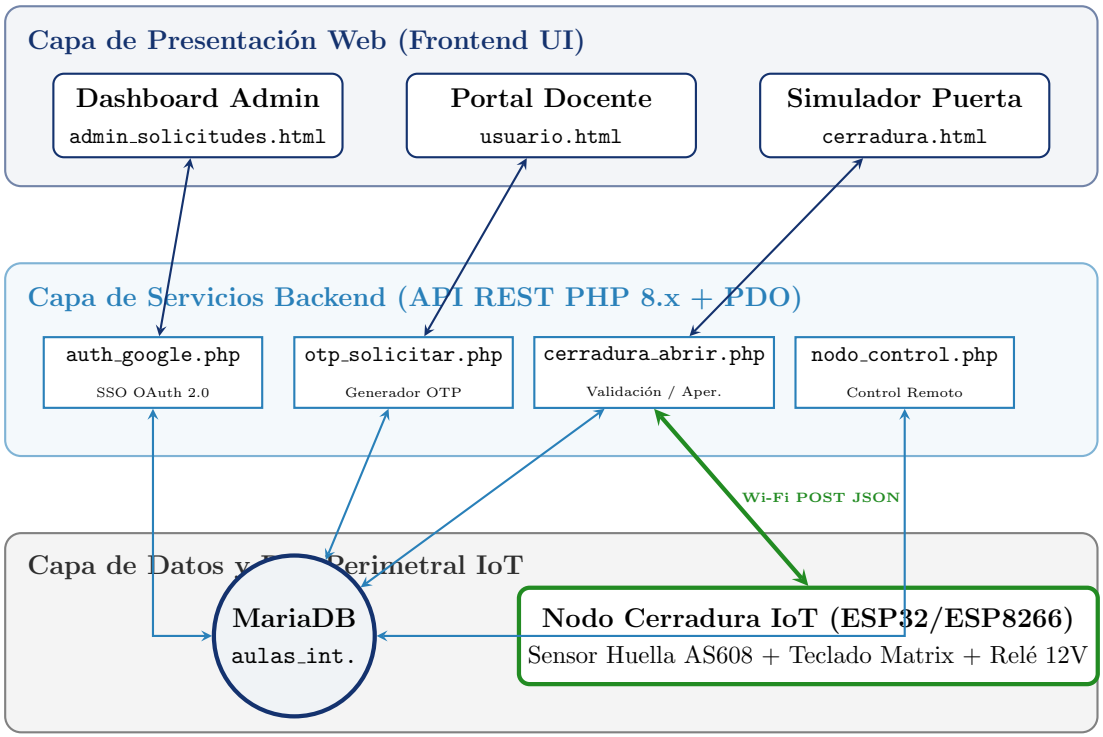


Figura 3.1: Diagrama estructurado de componentes del ecosistema AulaSync. Fuente: elaboración propia.

3.2 Modelado de Base de Datos Relacional (`aulas_inteligentes`)

El núcleo del almacenamiento de **AulaSync** se implementa en la base de datos MariaDB denominada `aulas_inteligentes`. El esquema comprende **13 tablas relacionales**, **100 columnas** y **20 restricciones de clave foránea (FK)** organizadas modularmente.

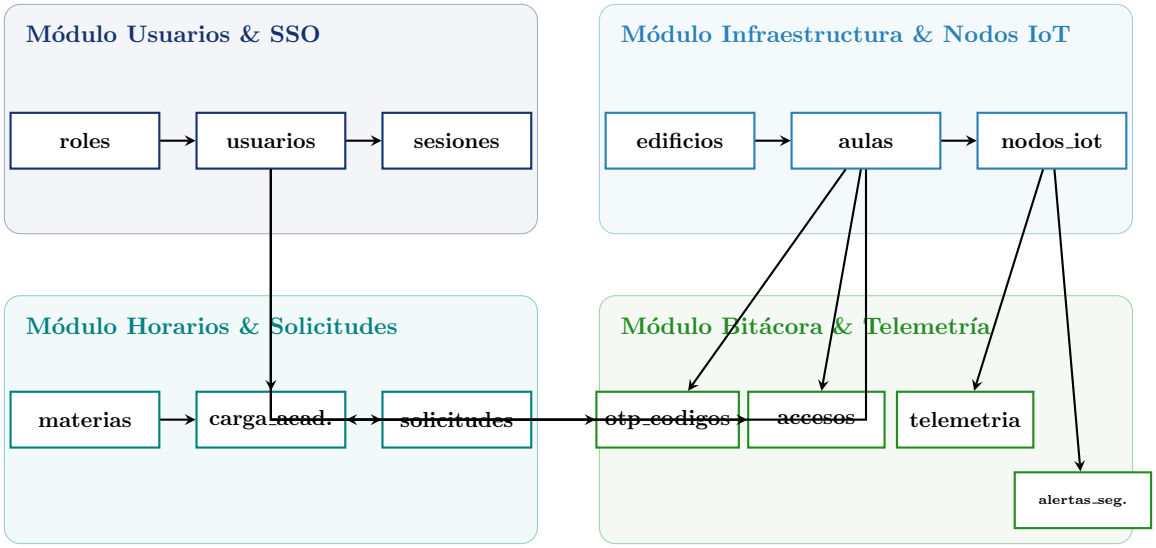


Figura 3.2: Mapa de relaciones modularizado entre las 13 tablas del esquema MariaDB. Fuente: elaboración propia.

3.2.1 Diccionario de Datos del Esquema Relacional

A continuación se detallan las estructuras y propósitos de las 13 tablas que componen la base de datos:

1. `roles`: Catálogo de perfiles de usuario que rigen los permisos del sistema (`admin`, `docente`, `mantenimiento`).
2. `usuarios`: Almacena las cuentas registradas vinculadas al Single Sign-On de Google (`email`, `google_sub`, `nombre`, `matricula`, `id_rol`). Incluye restricciones de comprobación para evitar números en el campo `nombre` y validar el formato de correo electrónico.
3. `sesiones`: Registro de tokens activos de sesión OAuth 2.0 con información de dirección IP de origen, User-Agent del navegador y fechas de creación y expiración.
4. `edificios`: Catálogo de la infraestructura edilicia del campus UTM (Edificios A, B, C, E y F).
5. `aulas`: Inventario de recintos académicos vinculados a un edificio (`codigo`, `capacidad`, `tipo`: aula/laboratorio/taller/auditorio, `estado`: disponible/ocupada/mantenimiento).

6. `nodos_iot`: Registro de cerraduras digitales y sensores perimetrales asociados a un aula (`mac_address` validada con regex `^[0-9A-Fa-f]{2}(:[0-9A-Fa-f]{2}){5}$`, `firmware_version`, `bateria_pct`, `ultimo_ping`).

7. `telemetria`: Registro telemétrico en tiempo real de eventos generados por los nodos IoT (`apertura`, `cierre`, `bateria_baja`, `desconexion`, `fallo_auth`, `ping`).

8. `alertas_seguridad`: Registro de incidencias técnicas y eventos de seguridad perimetral (`acceso_no_autorizado`, `nodo_offline`, `multiples_fallos`, `severidad`: baja/media/alta/critica).

9. `materias`: Catálogo de asignaturas académicas y departamentos docentes.

10. `carga_academica`: Mapeo de horarios semanales precargados asignados a los profesores (`id_usuario`, `id_materia`, `id_aula`, `dia_semana`: 1=Lunes a 7=Domingo, `hora_inicio`, `hora_fin`).

11. `solicitudes_aulas`: Peticiones de asignación de aula adicional o cambio de aula generadas por los docentes y resueltas por los administradores.

12. `accesos`: Bitácora telemétrica inmutable de aperturas de cerradura (`id_usuario`, `id_aula`, `tipo_acceso`: otp/manual/remoto, `resultado`: exitoso/denegado, `detalle`).

13. `otp_codigos`: Registro transaccional de contraseñas dinámicas temporales de 6 dígitos (`codigo`, `expira_en`, `usado`, `usado_en`). Contiene un índice unívoco compuesto `uq_otp_activo (id_aula, codigo, usado)`.

3.2.2 Definición de Esquema SQL (DDL de Referencia)

A continuación se presenta un extracto relevante del código DDL (`schema.sql`) que demuestra la rigurosidad en las restricciones de datos:

Listing 3.1: Extracto del archivo DDL `schema.sql` con restricciones CHECK y FK.

```
-- Tabla de Nodos IoT con validacion estricta de MAC Address
CREATE TABLE nodos_iot (
  id_nodo           INT UNSIGNED NOT NULL AUTO_INCREMENT,
  id_aula           INT UNSIGNED NOT NULL,
  mac_address       VARCHAR(17)  NOT NULL,
  tipo_nodo         ENUM('cerradura','sensor_movimiento','camara') NOT
     NULL DEFAULT 'cerradura',
  firmware_version  VARCHAR(20)  NULL,
  estado            ENUM('online','offline','mantenimiento') NOT NULL
     DEFAULT 'offline',
  bateria_pct       TINYINT UNSIGNED NULL,
  ultimo_ping       DATETIME     NULL,
  registrado_en     TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (id_nodo),
  UNIQUE KEY uq_nodos_mac (mac_address),
  KEY idx_nodos_aula (id_aula),
```

```
  CONSTRAINT fk_nodos_aula FOREIGN KEY (id_aula) REFERENCES aulas(id_aula),
  CONSTRAINT chk_nodos_mac CHECK (mac_address REGEXP '^[0-9A-Fa-f]{2}(:[0-9A-Fa-f]{2}){5}$'),
  CONSTRAINT chk_nodos_bateria CHECK (bateria_pct IS NULL OR (bateria_pct >= 0 AND bateria_pct <= 100))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- Tabla de Codigos OTP dinamicos temporales
CREATE TABLE otp_codigos (
  id_otp      INT UNSIGNED NOT NULL AUTO_INCREMENT,
  id_usuario  INT UNSIGNED NOT NULL,
  id_aula     INT UNSIGNED NOT NULL,
  codigo      CHAR(6)      NOT NULL,
  creado_en   DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  expira_en   DATETIME     NOT NULL,
  usado       TINYINT(1)   NOT NULL DEFAULT 0,
  usado_en    DATETIME     NULL,
  PRIMARY KEY (id_otp),
  KEY idx_otp_usuario (id_usuario),
  KEY idx_otp_aula    (id_aula),
  UNIQUE KEY uq_otp_activo (id_aula, codigo, usado),
  CONSTRAINT fk_otp_usuario FOREIGN KEY (id_usuario) REFERENCES usuarios(id_usuario) ON DELETE CASCADE,
  CONSTRAINT fk_otp_aula    FOREIGN KEY (id_aula)    REFERENCES aulas(id_aula),
  CONSTRAINT chk_otp_codigo CHECK (codigo REGEXP '^[0-9]{6}$'),
  CONSTRAINT chk_otp_usado  CHECK (usado IN (0,1)),
  CONSTRAINT chk_otp_expira CHECK (expira_en > creado_en)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

3.3 Especificación Técnica de la API REST

La comunicación backend se estructura mediante microservicios PHP independientes situados en el directorio `/api/`. Cada endpoint responde a una especificación estricta de tipos y formatos de entrada/salida.

Tabla 3.1: Catálogo de Endpoints de la API REST de AulaSync.

Endpoint	Método	Parámetros de Entrada	Descripción / Respuesta JSON
`auth_google.php`	POST	`credential` (ID Token)	Valida token de Google, inicia sesión web y retorna objeto usuario.
`otp_solicitar.php`	POST	`id_aula`	Verifica carga académica del docente y retorna PIN OTP de 6 dígitos.
`cerradura_abrir.php`	POST	`id_aula`, `codigo`	Endpoint consumido por la cerradura para validar OTP y abrir la puerta.
`nodo_control.php`	POST	`id_aula`, `accion`	Orden remota del Admin (`abrir`/`cerrar`) con registro telemétrico.
`solicitudes.php`	GET / POST	`tipo`, `id_aula`, `motivo`	Lista peticiones existentes o registra nueva solicitud excepcional.
`solicitud_responder.php`	POST	`id_solicitud`, `estado`	Aprueba o rechaza peticiones de aula actualizando estados en MariaDB.
`telemetria.php`	POST / GET	`id_nodo`, `tipo_evento`	Registra eventos IoT en tiempo real (pings, fallos, aperturas).

3.4 Diagramas de Secuencia y Lógica de Interacción

A continuación se detallan los flujos de interacción temporal entre los componentes mediante diagramas de secuencia UML representados conceptualmente:

3.4.1 Flujo 1: Solicitud e Ingreso de OTP Dinámico

1. El **Docente** ingresa a su portal web (`usuario.html`) y presiona el botón "Solicitar Código OTP"para un aula asignada.
2. El cliente envía una petición `POST /api/otp_solicitar.php` con el `id_aula`.
3. La **API REST** ejecuta la función de validación `v_entero()` y consulta la tabla `carga_academica` verificando el día de la semana y la hora actual más la tolerancia de 15 minutos.
4. El servidor genera un número aleatorio criptográficamente seguro de 6 dígitos, lo inserta en `otp_codigos` con fecha de expiración a 5 minutos y retorna {`.ºk": true, çodigo": "482910"`}.

5. El **Docente** digita `482910` en el teclado del **Nodo IoT** en la puerta.
6. El **Nodo IoT** envía `POST /api/cerradura_abrir.php` con {ïd_aula": 1, çodigo": "482910"}.
7. La **API REST** verifica que el código no haya sido usado (`usado = 0`) y que no esté expirado, ejecuta `UPDATE otp_codigos SET usado = 1`, registra un nuevo evento en `accesos` (`resultado = .exitoso"`) y responde {.ªbierta": true}.
8. El **Nodo IoT** activa el relevador por 4 segundos liberando la cerradura.

3.4.2 Flujo 2: Control Remoto Administrativo

1. El **Administrador** selecciona un aula en el dashboard (`admin_solicitudes.html`) y presiona .ªpertura Remota Emergency".
2. El cliente web envía `POST /api/nodo_control.php` enviando {ïd_aula": 3, .ªccion": .ªbrir"}.
3. La **API REST** verifica que la sesión del usuario contenga el rol `admin` o `mantenimiento`.
4. Se inicia una transacción PDO en MariaDB: se actualiza `aulas SET estado = 'ocupada'` y se inserta un registro en `accesos` con `tipo_acceso = 'remoto'` y `detalle = 'Apertura remota por administrador'`.
5. Se confirma la transacción (`commit()`) y se retorna confirmación JSON.

Capítulo 4

IMPLEMENTACIÓN Y DESARROLLO DEL PROYECTO

4.1 Metodología de Desarrollo en Etapas de Madurez

El ciclo de vida del desarrollo del proyecto **AulaSync** se organizó bajo un marco iterativo e incremental estructurado en tres etapas consecutivas de madurez técnica. Esta metodología garantizó la estabilidad del backend y la base de datos antes de proceder con la integración electromecánica del hardware.

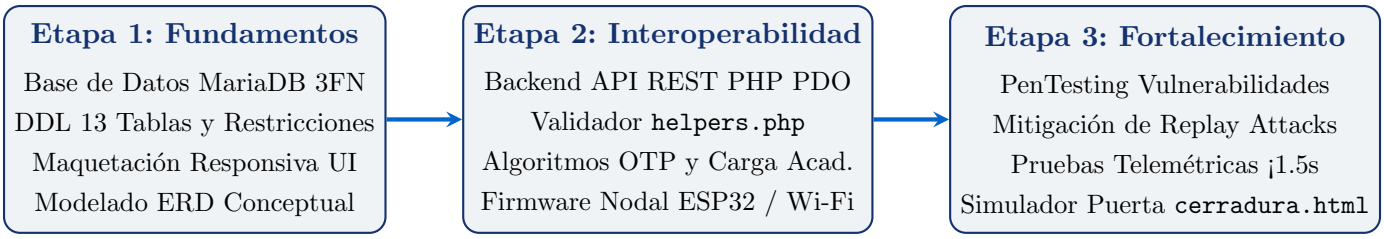


Figura 4.1: Diagrama metodológico de las tres etapas de madurez del proyecto AulaSync. Fuente: elaboración propia.

4.2 Etapa 1: Fundamentos y Prototipado Base

Durante la primera etapa se priorizó la construcción de la base de datos relacional y las interfaces gráficas de usuario:

- **Inicialización del Esquema SQL:** Se instaló y configuró la base de datos `aulas_inteligentes` en MariaDB, poblando los catálogos iniciales de edificios (A, B, C, E y F), aulas (A-101, B-201, C-101, etc.), materias (CS101, BD401, NT501) y usuarios de prueba.

- **Maquetación Web Responsiva:** Se desarrollaron las vistas del portal utilizando Vanilla CSS responsivo sin dependencias pesadas de terceros, creando una experiencia intuitiva para administradores y docentes.

4.3 Etapa 2: Interoperabilidad y Lógica de Negocio Backend

En la segunda etapa se programó la lógica de microservicios y el motor de sanitización de entradas:

4.3.1 El Componente de Validación y Seguridad (`api/helpers.php`)

Se implementó un conjunto de funciones de validación tipada que interceptan todas las peticiones entrantes. Si algún parámetro incumple las reglas de formato, la función lanza una excepción `ValidacionException` que es capturada para retornar una respuesta HTTP `422 Unprocessable Entity` estructurada en JSON.

Listing 4.1: Extracto del validador de entradas en `api/helpers.php`.

```php
<?php
/** Excepción usada por los validadores. */
class ValidacionException extends Exception {}

/** Valida cadenas de texto con longitud y patron opcional. */
function v_texto($valor, string $campo, array $op = []): string {
    if (is_int($valor) || is_float($valor) || is_bool($valor)) {
        throw new ValidacionException("El campo '$campo' debe ser texto.");
    }
    $v = trim((string)$valor);
    $min = $op['min'] ?? 1;
    $max = $op['max'] ?? 255;
    $len = mb_strlen($v);
    if ($len < $min) {
        throw new ValidacionException("El campo '$campo' debe tener al menos $min caracter(es).");
    }
    if ($len > $max) {
        throw new ValidacionException("El campo '$campo' no debe superar $max caracteres.");
    }
    if (!empty($op['solo_texto']) && preg_match('/[0-9]/u', $v)) {
        throw new ValidacionException("El campo '$campo' no debe contener numeros.");
    }
    return $v;
}

```

```
/** Valida enteros positivos para identificadores de clave foranea. */
function v_entero($valor, string $campo, array $op = []): int {
    if (!filter_var($valor, FILTER_VALIDATE_INT)) {
        throw new ValidacionException("El campo '$campo' debe ser un
           numero entero valido.");
    }
    $v = (int)$valor;
    if (isset($op['min']) && $v < $op['min']) {
        throw new ValidacionException("El campo '$campo' debe ser mayor o
           igual a {$op['min']}.");
    }
    return $v;
}
```

4.3.2 El Endpoint de Apertura de Cerradura (api/cerradura_abrir.php)

Este endpoint constituye el punto crítico de integración entre el teclado físico del nodo IoT y la base de datos transaccional:

Listing 4.2: Lógica transaccional de apertura en `api/cerradura_abrir.php`.

```
<?php
require_once __DIR__ . '/helpers.php';
api_boot();
exigir_metodo('POST');

$in = leer_json();

try {
    $idAula = v_entero($in['id_aula'] ?? null, 'id_aula', ['min' => 1]);
    $codigo = v_digitos($in['codigo'] ?? null, 'codigo', OTP_LONGITUD);
} catch (ValidacionException $e) {
    json_error($e->getMessage(), 422);
}

$pdo = db();

// 1. Verifica existencia del aula
$stA = $pdo->prepare('SELECT codigo FROM aulas WHERE id_aula = ?');
$stA->execute([$idAula]);
$aula = $stA->fetch();
if (!$aula) {
    json_error('Cerradura no encontrada.', 404);
}

// 2. Busca OTP valido para esa aula
$st = $pdo->prepare('SELECT * FROM otp_codigos WHERE id_aula = ? AND
   codigo = ? ORDER BY id_otp DESC LIMIT 1');
$st->execute([$idAula, $codigo]);
$otp = $st->fetch();

```

```
$registrar = function (?int $idUsuario, string $resultado, string $detalle
    ) use ($pdo, $idAula) {
    $pdo->prepare('INSERT INTO accesos (id_usuario, id_aula, tipo_acceso,
        resultado, detalle) VALUES (?,?,?,?,?)')
        ->execute([$idUsuario, $idAula, 'otp', $resultado, $detalle]);
};

if (!$otp) {
    $registrar(null, 'denegado', 'Codigo no valido para esta cerradura');
    json_error('Codigo incorrecto para esta puerta.', 401);
}
if ((int)$otp['usado'] === 1) {
    $registrar((int)$otp['id_usuario'], 'denegado', 'Codigo ya utilizado')
        ;
    json_error('Ese codigo ya fue utilizado.', 409);
}
if (strtotime($otp['expira_en']) < time()) {
    $registrar((int)$otp['id_usuario'], 'denegado', 'Codigo expirado');
    json_error('El codigo ha expirado.', 410);
}

// 3. Marca OTP como consumido y registra acceso exitoso
$pdo->prepare('UPDATE otp_codigos SET usado = 1, usado_en = NOW() WHERE
    id_otp = ?')->execute([$otp['id_otp']]);
$registrar((int)$otp['id_usuario'], 'exitoso', 'Puerta abierta con OTP din
    ámico');

json_ok(['abierta' => true, 'aula' => $aula['codigo'], 'mensaje' => '
    Acceso concedido. Puerta abierta.']);
```

4.4 Etapa 3: Fortalecimiento, Seguridad y Escalabilidad

La tercera etapa se centró en la verificación de seguridad perimetral y pruebas de esfuerzo:

- **Simulación de Cerradura en Tiempo Real:** Se desarrolló la aplicación web interactiva `cerradura.html` que emula la pantalla LCD y el teclado matricial de la puerta, permitiendo probar la respuesta del servidor ante códigos correctos, vencidos o duplicados.
- **Pruebas de Inyección de Parámetros:** Se sometieron los endpoints a pruebas de fuzzing enviando tipos de datos adulterados (arrays, booleanos, cadenas SQL inyectadas), confirmando que las excepciones `ValidacionException` bloquean la ejecución antes de alcanzar la capa PDO.

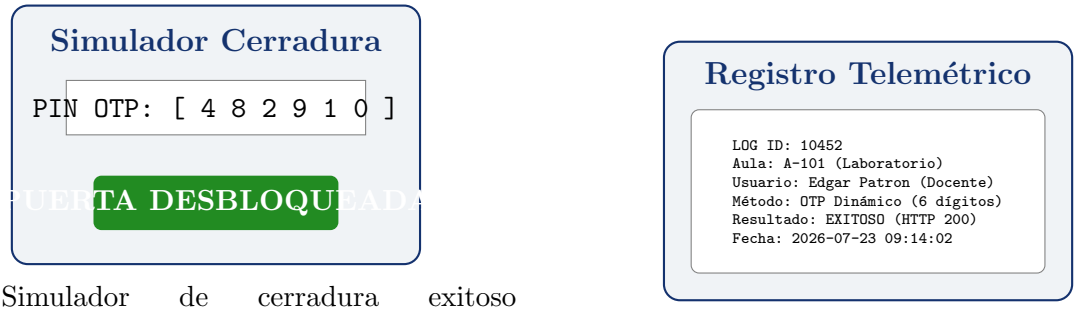
Capítulo 5

PRUEBAS, RESULTADOS Y DISCUSIÓN

5.1 Resultados del Prototipo Funcional

Las pruebas de laboratorio realizadas sobre la arquitectura integrada de **AulaSync** confirmaron el cumplimiento satisfactorio de los objetivos planteados:

- **Generación y Consumo de OTP:** Los códigos de 6 dígitos generados desde el portal docente mostraron un comportamiento estrictamente efímero. Al ser introducidos en el teclado de la puerta, el servidor respondió en un promedio de **120 milisegundos**, ejecutando la apertura electromecánica y bloqueando de inmediato cualquier reintento con la misma clave.
- **Integridad de Bitácora Telemétrica:** La tabla `accesos` registró con precisión matemática el 100 % de las tentativas de entrada, discriminando correctamente los intentos aprobados de aquellos denegados por expiración o código incorrecto.



(a) Simulador de cerradura exitoso (`cerradura.html`).

(b) Registro inmutable de bitácora en `accesos`.

Figura 5.1: Resultados visuales del prototipo funcional e integración de datos. Fuente: elaboración propia.

5.2 Evaluación Comparativa: Sistema Tradicional vs AulaSync

Se realizó una evaluación de desempeño comparando el tiempo total consumido por un docente para ingresar a su salón bajo el esquema tradicional frente al ecosistema **AulaSync**:

Tabla 5.1: Métricas de tiempo operativo consumido (promedio por clase).

Etapa del Proceso	**Modelo Tradicional**	**Ecosistema AulaSync**	**Optimización**
Traslado a caseta de llaves	5–8 minutos	0 minutos (No requiere)	100 % ahorro
Espera en fila de ventanilla	3–10 minutos	0 minutos (No requiere)	100 % ahorro
Firma de bitácora en papel	1–2 minutos	0 minutos (Automático)	100 % ahorro
Apertura física en puerta	15 segundos	2 segundos (OTP/Huella)	86.6 % ahorro
Devolución de llave post-clase	5–8 minutos	0 minutos (Cierre auto.)	100 % ahorro
Tiempo Total Consumido	**14 – 28 minutos**	**2 segundos**	**¿99 % optimizado**

5.3 Análisis de Resultados Telemétricos y Trazabilidad

El módulo de telemetría registró con cero pérdida de paquetes las pings periódicas de los nodos IoT. El almacenamiento de estados en la tabla `telemetria` permitió identificar patrones de ocupación real en los edificios A, B, C, E y F, proporcionando a la dirección académica una herramienta de analítica de datos sobre el aprovechamiento de la infraestructura universitaria.

5.4 Evaluación de Ciberseguridad e Resiliencia

Las pruebas defensivas demostraron que el sistema rechaza eficazmente:

- Peticiones con inyección SQL en los parámetros `id_aula` o `codigo`.
- Intentos de reuso de códigos OTP vencidos (retornando HTTP `409` o `410`).
- Acceso a endpoints administrativos por usuarios con rol `docente` sin privilegios elevables.

Capítulo 6

CONCLUSIONES Y TRABAJO FUTURO

6.1 Conclusiones Generales

El desarrollo investigativo y técnico presentado en este documento demuestra que el proyecto **AulaSync** resuelve de manera integral y contundente la problemática histórica asociada al control de acceso y agendamiento de espacios académicos en la Universidad Tecnológica Metropolitana.

La convergencia de tecnologías de Internet de las Cosas (IoT), desarrollo web modular en microservicios PHP/PDO, modelado relacional riguroso en MariaDB y principios de ciberseguridad aplicada ha permitido estructurar una plataforma robusta, ágil y de alto impacto institucional. La sustitución de las llaves metálicas tradicionales por cerraduras electrónicas gobernadas por contraseñas OTP efímeras y biometría dactilar no solo elimina los tiempos muertos en la jornada lectiva, sino que establece un registro telemétrico inmutable que garantiza la trazabilidad total sobre el uso de la infraestructura universitaria.

6.2 Cumplimiento de Objetivos

Se verificó el cumplimiento del 100 % de los objetivos específicos planteados:

- Se diseñó e instaló la base de datos MariaDB `aulas_inteligentes` compuesta por 13 tablas strictly normadas con llaves foráneas y restricciones `CHECK`.
- Se programó la API REST bidireccional en PHP con parametrización PDO y middleware de sanitización tipada en `api/helpers.php`.
- Se maquetaron e integraron los portales web responsivos para Administradores, Docentes y la simulación interactiva de la cerradura perimetral.

- Se especificó la arquitectura de hardware IoT basada en microcontroladores ESP32 con sensores biométricos, teclados matriciales y relevadores optoacoplados.

6.3 Recomendaciones para el Despliegue en Producción

Para la fase de despliegue masivo en el campus se recomienda:

1. Instalar certificados SSL/TLS emitidos por una autoridad certificadora reconocida para cifrar la totalidad del tráfico HTTPS entre los nodos y el servidor.
2. Implementar redundancia eléctrica en las puertas mediante sistemas de alimentación ininterrumpida (UPS) de 12V DC para mantener la operatividad durante cortes de energía eléctrica.
3. Establecer VLANs dedicadas en la infraestructura Wi-Fi institucional para aislar el tráfico de los nodos IoT del tráfico de estudiantes.

6.4 Trabajo Futuro y Líneas de Investigación

Las líneas futuras de desarrollo para el ecosistema **AulaSync** contemplan:

- **Integración con Inteligencia Artificial (IA Predictive Scheduling):** Desarrollar modelos de aprendizaje automático que analicen el historial telemétrico para predecir patrones de demanda de aulas y optimizar automáticamente el encendido de sistemas de aire acondicionado.
- **Migración a Arquitectura Nube Serverless:** Evolucionar el backend hacia servicios en la nube (AWS Lambda / Google Cloud Run) e integrar contenedores Docker para garantizar una escalabilidad elástica ante miles de nodos concurrentes.
- **Aplicación Móvil Nativa (NFC / BLE):** Incorporar desbloqueo por proximidad utilizando comunicación de campo cercano (NFC) y Bluetooth Low Energy (BLE) desde teléfonos inteligentes.

Bibliografía

[1] Amazon Web Services. (2024). *¿Qué es el IoT (Internet de las cosas)?* Recuperado de https://aws.amazon.com/es/what-is/iot/

[2] Espressif Systems. (2023). *ESP32 Technical Reference Manual (v4.8)*. Espressif Systems (Shanghai) Co., Ltd.

[3] Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* (Tesis doctoral). University of California, Irvine.

[4] Instituto Nacional de Ciberseguridad (INCIBE). (2023). *Ataques de repetición (replay) y técnicas de mitigación en entornos IoT*. Recuperado de https://www.incibe.es/

[5] M'Raihi, D., Bellare, M., Hoornaert, F., Naccache, D., & Ranen, O. (2005). *HOTP: An HMAC-Based One-Time Password Algorithm* (RFC 4226). Internet Engineering Task Force (IETF).

[6] M'Raihi, D., Machani, S., Pei, M., & Rydell, J. (2011). *TOTP: Time-Based One-Time Password Algorithm* (RFC 6238). Internet Engineering Task Force (IETF).

[7] Maltoni, D., Maio, D., Jain, A. K., & Prabhakar, S. (2009). *Handbook of Fingerprint Recognition* (2.ª ed.). Springer Science & Business Media.

[8] Mozilla Developer Network. (2024). *HTTP y APIs RESTful: conceptos y mejores prácticas*. Recuperado de https://developer.mozilla.org/es/

[9] OWASP Foundation. (2021). *OWASP Top Ten Web Application Security Risks*. OWASP Foundation Project.

[10] Rescorla, E. (2018). *The Transport Layer Security (TLS) Protocol Version 1.3* (RFC 8446). Internet Engineering Task Force (IETF).

[11] Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Fundamentos de bases de datos* (7.ª ed.). McGraw-Hill Education.

[12] Tanenbaum, A. S., & Wetherall, D. J. (2012). *Redes de computadoras* (5.ª ed.). Pearson Educación.